Fuzz like a Pro: Using Auditor Knowledge to Detect Financial Vulnerabilities in Smart Contracts

Bowen Cai*, Weiheng Bai*, Youshui Lu†, Kangjie Lu*
* University of Minnesota - Twin Cities † Xi'an Jiaotong University
{cai000254, bai00093}@umn.edu, {yolu6176}@uni.sydney.edu.au, {kjlu}@umn.edu

***Abstract*—Blockchain technology supports digital currency applications through Ethereum's smart contracts, enabling various functionalities but susceptible to security vulnerabilities. Notably, smart contracts are susceptible to Monetary Exploit Vulnerabilities (MEVs), which result in significant financial losses, evidenced by the billions of dollars lost to hacks in recent years. Traditional fuzzing tools, focused on underlying vulnerabilities such as opcode and stack patterns, fail to address complex, high-level MEVs that involve intricate interactions between contract functions and states. This paper introduces FuzzLaPRO (Fuzz like a PRO), an innovative fuzzing tool designed to understand and simulate smart contract logic to detect MEVs effectively. FuzzLaPRO leverages a Natural Language Processing (NLP) model, trained on a corpus of expert auditing reports and proofs of concept, to generate risk-aware function sequences that target MEVs. Additionally, we address the limitations of existing tools in handling distributed applications (Dapps) by implementing dynamic version discovery and implicit contract bridging. This allows FuzzLaPRO to adapt to various Ethereum Virtual Machine (EVM) versions and interact with implicitly deployed contracts, enhancing its applicability to real-world scenarios. We evaluated FuzzLaPRO on a dataset of 15,132 small and medium-sized smart contracts and 23 Dapps, discovering 79 new vulnerabilities at the smart contract level and 4 at the Dapp level, surpassing state-of-the-art fuzzers in both coverage efficiency and bug discovery rate. This work advances the smart contract security field by introducing a targeted, efficient fuzzing approach and sets a foundation for further innovations in automated security testing tools.**

I. Introduction

Blockchain technology, a decentralized and distributed ledger, is the foundation for digital currencies such as Bitcoin and Ethereum. A pivotal feature of Ethereum's blockchain is the smart contract[8], a self-executing program that significantly enhances the blockchain's functional diversity. These smart contracts are widely employed across various digital currency applications, including decentralized exchange applications, lending protocols, and digital insurance protocols. By design, smart contracts are immutable; although they can be upgraded, such modifications are often costly [8].

Smart contracts are highly susceptible to security breaches due to their intrinsic connection with monetary transactions and public accessibility. The risks have manifested dramatically in recent years, with over a billion US dollars lost to hacks, including $3.7 billion in 2022 and $1.7 billion in 2023 [37]. Consequently, addressing security vulnerabilities in smart contracts has emerged as a paramount concern within the blockchain industry and academia.

In the past few years, fuzzing has emerged as particularly effective, exploiting real-world environments to identify security vulnerabilities in smart contracts. The popular basic fuzzing methods like Choi et al.[6] and Liu et al.[25] propose to match the vulnerable pattern of the opcode and stack and use the symbolic execution to discover the underlying vulnerabilities. More recently, based on existing detection methods, some researchers propose the machine/deep learning model directed fuzzer to improve the efficiency in covering code, like ILF[14], Skyfire[39] and Smolka et al.[34]

The existing tools contribute greatly to smart contract security, eliminating most classical underlying vulnerabilities, like re-entrance, overflow, delegate call, etc. However, real-world cases show multitudinous vulnerabilities still emerge in the smart contract. Once we look into some auditing reports[19, 20], we can find that most vulnerabilities are not related to these traditional underlying vulnerabilities. Some of them are caused by the defective design of the price oracle[15], interest/swap rate mechanism[18], flash loan mechanism[16] or sometimes the combination of them. These cases are much more complex than the existing tools can handle. Meanwhile, these strange and sophisticated vulnerabilities have caused most of the money loss, even 100+ million per attack. We define vulnerabilities that directly result in monetary loss as Monetary Exploit Vulnerabilities. To cover all cases of monetary loss in smart contracts, we categorize them into four types: 1. Transfer Non-Arrival - This occurs when the target user does not receive tokens sent by the sender during a transaction; 2. Balance Non-Conservation - This occurs when the total balance of normal users in the transaction pool is not conserved; 3. Rate Stability - This occurs when the interest/exchange rate remains stable during a sequence of transactions; 4. Value Stability - This occurs when the contract's total value remains stable during a sequence of transactions.

By reflecting the auditing reports[20], we find the intensive manual review is the only efficient way to find this Monetary Exploit Vulnerability for now. The existing automatic tools are unreliable and even produce bunches of irrelevant information, confusing the auditors[4], because those vulnerabilities are high-level, logic-related.

The difficulty of this task is obvious: Some Monetary Exploit Vulnerability is only triggered by long and strictly designed function sequences. For human beings, to fully audit a smart contract requires the auditor to have a profound understanding of the smart contract and carefully analyze the relationship between different contracts[36]. It takes at least 2 weeks for a veteran auditor to review a smart contract, and the cost is very high. For the newbie auditors, it's even harder;

2024 International Conference on Meta Computing (ICMC) | 979-8-3503-5599-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICMC60390.2024.00032

979-8-3503-5599-4/24/$31.00 ©2024 IEEE
DOI 10.1109/ICMC60390.2024.00032

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

sometimes they spend more than 1 month finding nothing[13].

Though there are some **dynamic tools** to detect these types of vulnerability by examining the fund flow, for example, some Ethereum transaction spectating tools[41, 42], they are mostly post-verification tools. Specifically, they only examine the previously happened exploitation. They surely cannot prevent exploitation, and thus, money loss is inevitable.

Another type of tool is the **formal verification tools**, which can detect the vulnerability beforehand. Working with human testers, they can detect some logic-related vulnerabilities[2, 24, 30, 32]. However, the scalability is so limited that the specific verification statement can not be applied to multiple smart contracts, and the verification statement cannot be generated automatically. In a word, it costs too much human resources and is not scalable.

The existing **fuzzer tools** cannot detect the Monetary Exploit Vulnerability. The vulnerability detection method of the existing fuzzer mostly only focuses on opcode and stack[6, 25], using pattern matching and symbolic execution[11, 35], to discover some underlying vulnerabilities, which is easy to apply and many State-of-the-Art fuzzer tools are using this method[14, 34, 39]. But for Monetary Exploit Vulnerability, it is not that simple. We have to account for the balance of the account, the money transfer, the interest rate, and the relationship between different tokens. None of the existing fuzzer tools can achieve this. Although some fuzzers[5] can be customized for this purpose, they cannot be scaled to contracts other than the targeted one and are also inefficient and inaccurate.

Thus, the current situation with tools is that most of them cannot understand the logic of the code and the relationship among contracts. Thus, they are not able to detect the Monetary Exploit Vulnerability. The spectacular amount of bounty is the best proof of the importance and difficulty of this problem[19] - if the tools can detect them, the bounty will not ask for help from the human auditors. On the other hand, the Monetary Exploitation Vulnerability is so lucrative by definition that attackers are willing to spend countless hours exploiting it, making this an exhausting war between the auditors and the attackers.

In summarizing the existing problem, the key to the question is to help the fuzzer understand the smart contract and the monetary exploitation vulnerability behind the code so it can generate a helpful function sequence to hit the vulnerability.

Thus, we propose the Nature Language Driven Fuzzer, named FuzzLaPRO(Fuzz like a PRO), to detect the Monetary Exploit Vulnerability, which can understand the smart contract logic to generate useful function sequences and is capable of discovering complex Monetary Exploit Vulnerability. To teach the fuzzer to understand smart contracts and generate the proper sequence of function calls. And we need to solve two main challenges:

1) There is no existing benchmark dataset constructed for the logic of the smart contract, how to collect and organize human's understanding of the smart contract logic and compose the dataset for the fuzzer to learn.
2) How to build the model trained on this dataset composed of the logic of the smart contract and the corresponding it with the codebase.

First, to address the challenges above and to organize human understanding of the smart contract, we collect the auditing report written by proficient auditors, as some white-hat hackers exploit PoCs. Then, we build the corpus of smart contracts. We composed the corpus with the corresponding codebase as the training data. Secondly, to train the fuzzer to understand the smart contract logic, we build the NLP model, so after the training, the model can recommend the higher risk sequential according to the current smart contract and its running status. We can call the model the **Fuzzer Brain**.

Besides, deficiencies in the existing fuzzer paradigm and practicability challenges are emerging: The existing fuzzer tools cannot detect application-level smart contracts, i.e., Dapp, distributed application. Traditional fuzzer tools only focus on underlying vulnerabilities and don't need to account for multiple contracts. The content in the opcode and stack is all they need to detect the vulnerability. But for the Monetary Exploit Vulnerability, we have to account for multiple contracts as a logical whole because the Monetary Exploit Vulnerability is related to the transfer of the tokens, the interest rate of the currencies, and the relationship between different tokens. Those values are stored in different contracts. Fuzzer has to link those contracts and have them fully deployed, interacting with each other, to detect the Monetary Exploit Vulnerability.

We solve these problems from two aspects: contracts' version dynamical discovery and implicit deployed contracts bridging. For **the dynamical version discovery**, there are always contracts with different EVM versions in one Dapp. The traditional fuzzer uses a fixed system EVM version to obtain the abi interacting with the contract, which is not feasible for the Dapp. Thus, our fuzzer tool dynamically adjusts the abi version for the contracts by checking its compiled opcode swarm hash and then uses the compatible abi to interact with the contract. For **the implicit contract bridging**, in the current popular Dapp, the factory pattern is widely used. Thus, most of the contracts are deployed implicitly, especially the token contracts, which we have to interact with to detect the Monetary Exploit Vulnerability. Obviously, the existing fuzzer tools are not able to accomplish these because they only focus on single contracts, not accounting for factory pattern deployment, let alone the implicit deployed contracts. To solve this, we design the system to automatically generate the shade contract from the implicit deployed target and then interact with the shade contract to detect the Monetary Exploit Vulnerability.

Finally, to build the **prototype system**, we collected 441 bug review articles from the auditing website, accompanied with the corresponding smart contract codebase, to build the dataset. Then, we train our NLP model on this dataset and build the sequential recommendation model and the vulnerability checker. In the **evaluation**, we test our fuzzer on the dataset of smart contracts composed of 15,132 small and medium-sized smart contracts (single contracts) and 23 Dapp on Ethereum. As a result, at the smart contract level, our fuzzer discovered **79** new vulnerabilities, and at the Dapp level, our fuzzer discovered **4** new vulnerabilities, both of which are not covered by the existing fuzzer tools. Our fuzzing method outperforms SOTA by 20% on coverage efficiency for Dapp-level contracts. We are also faster than SOTA in discovering bugs, outperforming it in 6 out of 9 types of vulnerabilities.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Auditing Article PoC[17]

1	'Make a deposit to the FutureVault contract using **deposit** function Call'
2	'**createFYTDelegationTo** from a fresh account and specify the attacker address as the recipient. totalDelegationsReceived[_receiver] is being updated. Delegation needs to happen from the fresh account as we want to bypass the following check'
3	'Call **withdraw** function redeeming PTs for IBTs. Due to the logic error in the _beforeTokenTransfer(), the check for the amount being redeemed against the redeem balance plus delegated tokens is not run. An attacker received all PTs they deposited in step 1 while keeping the delegation amount'
4	'**Repeat** steps 1 to 3 multiple times'
5	'Wait a few periods for the inflated delegation to generate yield'
6	'**Mint** PTs and FYTs and call **withdraw** to redeem these tokens to IBT'

In summary, our main **contributions** are:

1) We propose the Fuzzer be able to understand the smart contract logic to generate a useful sequence of function calls, thus efficiently discovering complex Monetary Exploit Vulnerability.
2) We construct the key component, Fuzzer's Brain, a nature language model trained on the dataset of expert auditing articles and PoCs, to recommend the higher risk function sequences according to the smart contract code and its runtime status.
3) For better practicability, We solve deficiencies being unable to accomplish detection on application-level smart contracts (Dapp) of the existing fuzzer paradigm, using contracts' version dynamical discovery methods and implicit deployed contracts bridging methods.
4) We build the prototype system and validate our system. The validation dataset comprises 15,132 small and medium-sized smart contracts (single contracts) and 23 Dapp on Ethereum. As a functionality result, it discovered **79** new vulnerabilities in the smart contract level and **4** new vulnerabilities in the Dapp level. As an efficiency result, it outperforms the SOTA fuzzer tools.

II. Overview & Motivation

In this session, we will give a brief overview of our fuzzer. We introduce a motivating example to exemplify the inspiration and pipeline of our fuzzer, in which the existing tools and their strategies are compared with our FuzzLaPro method, embodying the essence of our fuzzer.

TABLE II: ABI Comparison Example. *The bold font means the function has similar attributes. The more similar the function, the more similar the smart contract, and the higher the probability they have the same vulnerability types.*

Smart Contract	Functions
Smart Contract A	**accrueInterest() -> void** getTotalAmount() -> uint256 **withdraw(address) -> uint256**
Smart Contract B	**accrueInterest() -> uint256** getAmount() -> uint256 **withdraw(address) -> uint256**
Smart Contract C	amount() -> uint256 getBalance(address) -> uint256 **withdraw(address) -> uint256**

Table I is part of an auditing report called Proof of Concept (PoC) on the auditing website. The PoC is a step-by-step guide to show how the vulnerability is exploited. The PoC requires the tester to interact with the smart contract to exploit the vulnerability rigorously. A proficient auditor spends at least 2 weeks understanding the smart contract and countless days discovering such a bug. Even with the PoC in hand, trying to reproduce the bug is still a challenging job. Thus, manual review is highly time-consuming and costly, but it is still the most essential way to discover Monetary Exploit Vulnerability. So many companies still pay a lot for bug bounty and auditing services.

In response to this urgent and vital demand, we propose to build a system to help understand the smart contract and continuously generate the test case to discover the vulnerabilities, working like a human professional auditor. The following example is an oversimplified example to show the basic idea of our system. In practice, each module is much more complex and has to be compatible with the real-world situation, so the details should be introduced in the later section IV and V.

*1) **NLP data extraction & sequential recommendation model(Brain)**:* Starting from the above, we want to understand auditors' thoughts on the smart contract to make the system professional. For example, having the PoC of table I in hand, it should extract the keywords from the PoC first; once the system finds the smart contract is similar to the one targeted by the PoC, it can generate the related test sequence to hit the vulnerability. Imitating the function call sequence:

`deposit()` → `createFYTDelegationTo()` → `withdraw()` → `LOOP 1-3` → `mint()` → `withdraw()`

it may generate another similar sequence:

`save()` → `delegateTo()` → `withdraw()` → `LOOP 1-3` → `mint()` → `withdraw()`

The similarity of the smart contract is quantified by comparing the abi of the smart contracts. A simple example of the abi comparison is shown in table II. Smart contracts A and B are more similar than smart contracts A and C. Thus, there is a higher probability that smart contracts A and B have identical types of vulnerability, and the system can generate a similar test sequence to validate the vulnerability.

*2) **Multi-contract DApp compatibility**:* Finally, to make the system practicable, we solve the problem of previous fuzzer tools that are not able to accomplish the detection on application-level smart contracts (Dapp) by proposing **contracts' version dynamical discovery**, and **implicit deployed contracts bridging**. The necessity to solve them is because the existing fuzzer tools are only focused on a single contract, not accounting for factory pattern deployment, thus not being able to test multiple contracts under one single environment and not being able to interact with the implicit deployed contracts. Our solution is to 1. dynamically adjust the abi version for the contracts by checking its compiled opcode swarm hash; 2. mock the implicit deployed contracts by generating the shade contract. The details of the solution will be introduced in the later section V.

This is an oversimplified example to show the basic idea of our system. In the later section, we will introduce the details of our system and the real-world situation. In section IV, we will introduce the formalized model of our fuzzer, including the Fuzzer's Brain - a NLP based function call sequence generator, the Fuzzer's Instinct - a Markov process-based self-study strategy, and the Fuzzer's Eye - the formal definition of 4 types of Monetary Exploit Vulnerability and the checker. In section V,

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

we will introduce the system model implementation, including the system's framework and workflow, the construction of the information extraction, the sequential recommendation, and the vulnerability detection modules. Also, the solution to the practicability challenge for multi-contract DApp compatibility. In section VI, we will introduce the experiment result, the evaluation of our system, and a study case to show the real-world situation. In section VII, we will introduce the related work and its limitations. We will conclude our work in section VIII.

III. Background

Smart contracts have been widely discussed recently, especially the Ethereum smart contract. Many security tools have been proposed, bug bounty programs have been launched, and many auditing companies have been established. Security tools are indispensable to making smart contracts more secure. However, because of their terrible false positive rate and low efficiency, they can only be used as a reference, and manual review is still the most crucial way to discover the vulnerability. Every time before the smart contract is launched, the issuer has to pay a large amount of money to the auditing company or bounty program to review the smart contract. Making security tools more efficient and accurate is always a problem that researchers are trying to solve.

A. Traditional Vulnerability

There is the complete list of the vulnerabilities; we can refer to the SWC[9], an official list maintained by the Ethereum Foundation to record the common vulnerabilities in the smart contract. Many classic vulnerabilities in the smart contract, such as Unhandled exceptions, Reentrance, Suicidal, Block State Dependency, Leaking, etc., have been widely recognized and studied. A lot of research and tools work on these vulnerabilities, such as Zeus[21], Mythril[28], Oyente[27], Echidna[12], SlitherSlither[10] and Scribble[40]. The mainstream method is the opcode and stack-based pattern matching, and depending on their execution methods, incorporating fuzzing, symbolic execution, or machine learning model. We elaborate on the existing tools in the section below.

B. Popular Detection Methods

The most popular detection methods can be categorized into three types: Static, Dynamic(fuzzer or online tool), and Formal Verification. The popular static tools includes Zeus[21], Mythril[28], Securify[38]. For the fuzzers, there are Oyente[27], Echidna[12], Slither[10], Scribble[40], ILF[14]. The difference exists in the code executor, where the static analyzer uses the solvers to cover the code in symbolic execution, and the fuzzer uses the random strategy to generate the real test case. That's why the fuzzer is always categorized as the dynamic analyzer. At the same time, their oracle(the vulnerability checker) is similar. Based on the opcode and stack or certain go-ethereum APIs, they check whether the code behaviors match the underlying vulnerability's pattern.

The dynamic analyzer also includes some online/real-time spectating tools[23, 29], which are post-verification tools and only confirm the exploitation after it happens, so it's out of our scope. For the formal verification tools, like F*[3] and Isabelle/HOL[1] etc., need manual verification statement written by the expert, and run on the automatic prover. Because it's a highly specified verification statement, it is not scalable, so it's out of our scope.

FuzzLaPRO's scope mainly focuses on detecting monetary exploit vulnerabilities more directly related to the most critical financial bugs. The detection methods are also totally different from the above, which are contracts ABI-based. Here, we should extract essential variables using ABI and then interact with the smart contract to validate the anomaly.

Limitation: As we mentioned, a common blind zone of the existing fuzzer is that they only check underlying vulnerabilities, which means their bug sanitizer only focuses on the opcode and stack. For example, a common reentrance bug checker is like this:

```
def check_reentrance(self, logger):
  has_transfer = False
  change_state = False
  pc1, pc2 = -1, -1

  for log in logger.logs:
    if log.op == CALL and int(log.stack[-3], 16) > 0
     and int(log.stack[-1], 16) > 0 :
        has_transfer = True
        pc1 = log.pc
    if log.op == SSTORE :
        change_state = True
        pc2 = log.pc

  pc_follow = ((pc1 != -1) and (pc2 != -1) and (pc1
    < pc2))

  return has_transfer and change_state and pc_follow
```

Listing 1: Traditional Reentrance Bug Checker

This is an ordinary reentrance bug checker. It detects a reentrance bug if there is a transfer and a state transition. If the transfer occurs before the state change, it is identified as a reentrance bug. However, this operation pattern is also ordinary elsewhere—it's very likely a piece of a safe program. For example, the token 'A' gets transferred, but the state transition is unrelated to token 'A' but to token 'B.' Thus, the result might be far from a bug, potentially incurring a high false-positive rate. This is also the current situation of fuzzers. The actual developers of smart contracts don't deserve to spend time on so much misinformation. In conclusion, our fuzzers should better only focus on those vital bugs. We call them Monetary Exploit Vulnerabilities.

C. Multiple Contracts Implementation

Inheriting the instinct of the traditional fuzzer, because the fuzzer only detects the underlying vulnerability, they don't need to account for the multiple contracts. We can check this feature quickly by running the popular static/fuzzing tools like Oyente[27], Mythril[28] or ILF[14]. Some might say they support multiple contracts, but it just means they are run respectively and not strictly linked together. The advantage is that it would be accessible in preparing deployments, and the disadvantage is so apparent:

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

Limitation: In the real world, critical financial bugs always occur across multiple contracts, involving multi-contract interaction. In many cases, the contracts are in different EVM versions. Also, token contracts are always implicitly deployed because of the prevailing factory pattern, an engineering challenge.

D. Popular Fuzzer's Strategies

To achieve our proposed detection, we exclude the Formal Verification tools and the Online Spectating tools from our options because the former is not scalable, and the latter is post-verification. Neither can effectively eliminate the critical financial issues in smart contracts. The static analyzer sounds like a good choice, but its low efficiency contradicts our idea of only focusing on the most essential parts. So, the fuzzer is the most suitable tool for our purposes.

Primarily, fuzzers use a random strategy, which is not efficient or accurate. Thanks to recent research, like SmarterContract and ILF[14, 33] started incorporating the machine learning model. The typical way is to sample the information from static analysis and then use the machine learning model to help the fuzzer decide. The reasoning behind them is to imitate the static analyzer's behavior and only test the most important branches of the code. These methods improved efficiency a lot. Some fuzzers use compiled bytecode incorporate CV model[22] to help the fuzzer make decisions. The idea is to transform the bytecode to the image and then use the CV model learning from the existing buggy code's bytecode image to help the fuzzer make decisions. This method also sounds very innovative.

Limitation: However, neither of the methods above can help us understand the smart contract's logic, which should start from the human vision. So, inspired by them, FuzzLaPRO proposes using the expert's auditing report to train the NLP model and then using the NLP model to help the fuzzer make a decision, which would be more consistent with our initial idea.

IV. Fuzzing Model

In this section, we will introduce the formalized model of our fuzzer, including the Fuzzer's Brain strategy - a rule-based function call sequence generator, the Fuzzer's Eye strategy - the formal definition of 4 types of Monetary Exploit Vulnerability and the checker, and the Fuzzer's Instinct strategy - a Markov process-based self-study strategy.

A. Fuzzing Strategy Definition

As the starting point, we define the fuzzer as a Random Variable, which reflects the current state of the fuzzer and the following function call to be made.

Definition 1 (Fuzzing Strategy): A Fuzzing Strategy is expressed as a Random Variable $\pi : \mathcal{T} \times \mathcal{T}^* \to \mathcal{F}$, where $\mathcal{T}$ is the collection of transaction t, $\mathcal{T}^*$ is the collection of transactions sequence $\vec{t} := (t_1, t_2, \ldots, t_n)$.

Here, π is a Random Variable mapping from transaction space to Function Space. So when we try to choose the following function, known as the previous sequence under specific contract C, we can express the Probability Density Function as $\pi : \mathcal{T}_C \times \mathcal{T}_C^* \to \mathcal{F}_C$. The π differs in different strategies.

Coercive Strategy: The coercive strategy is executed in the Brain module, the first strategy to be executed when the fuzzer is in the starting phase. The strategy is to make the fuzzer generate the sequence strictly following the rules constructed by the expert's auditing report.

Firstly, we define the rule based on the transaction's definition and power set.

Definition 2 (Rule): When $\mathcal{T}_r$ is the set of the transactions, and $\mathcal{T}_r^*$ is the power set of the transactions. A Rule is denoted as a transaction sequence $\vec{t_r}$, that

$$\vec{t_r} = (t_1^{'}, t_2^{'}, \ldots, t_n^{'}), \qquad \begin{matrix} \vec{t_r} \in \mathcal{T}_r^* \\ t_i^{'} \in \mathcal{T}_r, 0 < i \leq n \end{matrix} \tag{1}$$

Having the rule $\vec{t_r}$, we assume $\vec{t} = (t_1, t_2, \ldots, t_i)$ is the sequence we have generated for certain contract, and the transactions before $t_i \in \mathcal{T}_C$ have been decided. The next transaction t_{i+1} is selected by the following processes:

When $i = 0$, the strategy searches for the rule fit to this contract:

$$\pi(t_1 \mid \vec{t_r}) \sim \mathcal{U}_{t_0 \in \mathcal{T}_C} \tag{2}$$

This means it keeps randomizing with a random uniform distribution, with the i remaining at 0. Once $t_0 \in \mathcal{T}_r$, the fuzzing process is trapped into the coercive strategy, and i is set to 1.

When $i \neq 0$, the strategy is to strictly make the fuzzing process follow the rule. If $\exists t_{i+1} \in \mathcal{T}_C, t_{i+1} = t_{i+1}^{'}$:

$$\pi(t_1 \mid \vec{t_r}) \sim \mathbb{P}(t_{i+1} = t_{i+1}^{'}) = 1 \tag{3}$$

, while if $\nexists t_{i+1} \in \mathcal{T}_C, t_{i+1} = t_{i+1}^{'}$:

$$\pi(t_1 \mid \vec{t_r}) \sim \mathcal{U}_{t_{i+1} \in \mathcal{T}_C} \tag{4}$$

At the end of the rule, meaning $i = n$, we reset $i = 1$ so the fuzzing process restarted.

Markov Strategy: The Markov strategy is executed in the Instinct module, which is the secondary strategy to be executed when the fuzzer has been trained by the coercive strategy. The Markov strategy is to make the fuzzer generate the sequence following the Markov process.

The difference between the Markov and Coercive strategies is that the choice of the next function is no longer decided by the rule but by the current state of the process. The current state of the process is defined as $t \in \mathcal{T}_C$ here, the probability distribution of the following function is expressed as $\pi : \mathcal{T}_C \times \mathcal{T}_C \to \mathcal{F}$. We can define a Naïve Markov strategy as:

$$\pi(t_{i+1} | t_i) \sim \mathcal{U}_{t_{i+1} \in \mathcal{T}_C} \tag{5}$$

To improve it, we can replace its probability distribution to make it more likely to discover bugs. We use the Policy(t_i) to express the new probability distribution:

$$\pi(t_{i+1} | t_i) \sim \text{Policy}(t_i) \tag{6}$$

V. System Model

In this section, we present the system implementation model of FuzzLaPRO. Figure 1 shows the framework of the system represented in Model & Data Flow Diagram. The introduction is divided into five sections: Data Extraction, Sequential Recommendation Model (Brain), Vulnerability Detection Model (Eye), Self-study Fuzzing Strategy (Intuition), and Multi-contract DApp Compatibility. For the Brain and Intuition part, we will give the pseudo-code of the model because of its complexity.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

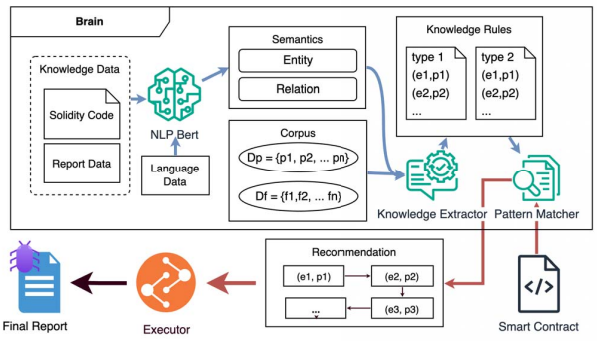


Fig. 1: The framework of the system model. *The system model comprises 3 main modules: the Brain, the Eye, and the Intuition. We use different color arrows to present different system workflows, where the blue arrow represents the inner process of each module, while the parallel blue line represents the smart contract ABI support for the models. The red arrow means the generated test sequences are mainly decided by the Brain, and the green arrow means the generated test sequence is mainly decided by Intuition.*

A. Data Preparation

The data preparation is the first few steps of the Brain module, which prepares the messy data collected from the Internet into the formalized data that can be used in the Brain module. The data preparation part is divided into 3 steps: NLP bert pre-training, Semantics Analysis, and Corpus Construction. We can see them in the left part of the figure 1, the Brain module. Because it is not the coral magic of the Brain's algorithm, we separate it from the Brain section V-B.

NLP Bert Data Processing: The main components of data processing are Our NLP Bert - our fine-tuned natural language processing model that can understand and analyze text data; the Knowledge Data - which includes reports and articles, providing rich background information and context for our system; the Language Data - the SciERC dataset[26] and our own customized dataset, to train our model.

The workflow of NLP Bert is a two-step process. The first step is to train the pre-trained model from the SciERC dataset with our customized dataset. This allows our model to understand and handle our specific language data. The second step is to extract semantic information from the Knowledge Data. This enables our system to understand and analyze the entities and relationships within our Knowledge Data.

From now on, we have extracted the semantic information from the Knowledge Data and generated it as entities and relationships. Specifically, the entities are the function names and parameter types, and the relationships are pairs of function names and parameter types.

The articles in Knowledge Data for NLP Bert can be described as a collection of nature language: $Bert_{in} = \{s_1 \dots s_n\}$. After processed by the trained NLP Bert, the output is described as $Bert_{out} = \{(E_1, R_1), (E_2, R_2) \dots (E_n, R_n)\}$

Each output result is an ordered sequence of entities and a sequence of relationships:
$E_i = \langle e_1, e_2, \dots, e_m \rangle, R_i = \langle (e_1, p_1), (e_2, p_2), \dots, (e_m, p_m) \rangle$

, and we have $p \in P, E \subseteq D_f, P \subseteq D_p, R \subseteq \mathcal{R}$, where P is the collection of parameter types, D_f is the collection of function names, D_p is the power set of parameter types, and $\mathcal{R}$ is the collection of relationships. A detailed description of the sets above can be found in the Corpus Construction section below.

Corpus Construction: The system incorporates a corpus manually constructed from general knowledge of smart contracts. This corpus is a collection of data that includes function names such as 'deposit,' 'withdraw,' 'transfer,' and so on. It also includes several parameter types, such as 'int' and 'string'.

This Corpus is denoted as D_f, P, and $\mathcal{R}$, where $\mathcal{R}$ is a subset of the Cartesian product of D_f and D_p, where $\mathcal{R} \subseteq D_f \times D_p$. The corpus D_f represents the collection of function names, P represents the collection of parameter types, and for convenience, we use D_p represent the Power Set of parameter types P in Figure 1. The corpus $\mathcal{R}$ represents the collection of pairs of function names and parameter types.

Rules Extraction: Rules extraction is the process of extracting the rules from the semantic information generated by the NLP Bert. The rules are the key information that the Brain module uses to generate the transaction sequences.

The last step we have the semantic information $Bert_{out}$, and the corpus D_f, P, and $\mathcal{R}$. Thus, the Knowledge Extractor matches the entities and relationships with the corpus's function names and parameter types and generates the rules. The rules are represented as a collection of transaction sequences: $\mathcal{T}_r = \{\vec{t_1}, \vec{t_2}, \dots, \vec{t_n}\}$, where $\vec{t_i} = \langle (f_1, p_1), \dots (f_m, p_m) \rangle = \langle t_1, \dots t_m \rangle$, which is generated from the semantic information $R_i = \langle (e_1, p_1), \dots (e_m, p_m) \rangle$. Each rule $\vec{t_i}$ reflects a certain type of contract is risky when the related transaction sequence is executed.

B. Sequential Recommendation Model

Now, we introduce the strategy of the recommendation model. The basic idea of the Coercive Strategy is to coercively generate the transaction sequence according to the rules produced by the NLP Bert. The strategy is contained in the Pattern Matcher of the Brain module in figure 1.

The detail of the Coercive Strategy is described in the Algorithm 1. We take the input as the contract's possible transaction $\mathcal{T}_C$, the current sequence $\vec{t}$, and the rules $\vec{t_r}$. The output is the next transaction t_{i+1}. Once we can match the current contract's status with the rules, we will follow the rules strictly to generate the transaction sequence. The Coercive Strategy is the cold-start strategy, the first phase of the recommendation model.

C. Multi-contract DApp Compatibility

In this section, we introduce the engineering challenge of the existing tools when implemented on Dapp-size contracts. Dapp-size contracts basically mean the application contains multiple contracts that need to be compiled and deployed in strict order, and the deployer needs to write scripts to do so. The two main challenges are the EVM version incompatibility and the contract implicit deployment.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Sequential Recommendation Model (Brain)

Require: $\mathcal{T_C}, \mathcal{T_C^*}, \vec{t} = (t_1, t_2, \ldots, t_i), \vec{t_r} = (t_1^{'}, t_2^{'}, \ldots, t_n^{'})$ ▷ The input includes the transaction collection, the current sequence, and the rules

Ensure: t_{i+1} ▷ The output is the next transaction

procedure POLICYCOERCIVE($\mathcal{T_C}, \mathcal{T_C^*}, \vec{t}, \vec{t}$)
  if Loop reaches limitation **then**
    return
  end if
  if $i = 0$ or $\nexists t_{i+1} \in \mathcal{T}_C, t_{i+1} = t_{i+1}^{'}$ **then**
    $\pi(t_{i+1} \mid \vec{t_r}) \sim \mathcal{U}_{t_1 \in \mathcal{T}_C}$ ▷ The random choice
    $\vec{t} = t_{i+1}$ ▷ Reset the sequence
  else ▷ The coercive strategy
    $\pi(t_{i+1} \mid \vec{t_r}) \sim \mathbb{P}(t_{i+1} = t_{i+1}^{'}) = 1$ ▷ $t_{i+1} = t_{i+1}^{'}$
    $\vec{t} = (t_1, t_2, \ldots, t_{i+1})$
  end if
  $s = \text{Execute}(t_{i+1})$ ▷ Execute the transaction
  PolicyCoercive($\mathcal{T_C}, \mathcal{T_C^*}, \vec{t}, \vec{t_r}$)▷ Recursion to next Loop
end procedure

EVM Version Compatibility*:* The traditional tools only support single contract testing, thus supporting only the single EVM version. Specifically, one test environment can only support one single swarm hash, e.g., `ipfs`, `bzzr0`, `bzzr1`, etc. The swarm hash is used to identify the contract's ABI, and the ABI is used to interact with the contract. Thus, the traditional tools cannot support multi-contract Dapp testing without recognizing multiple swarm hashes.

To solve this problem, we integrate the contract hash discovery function into our dynamic analyzer in the Eye module in figure 1. This workflow can be described as follows: 1. The dynamic analyzer searches the contract hash (swarm hash) from the end of the contracts' opcode; 2. The dynamic analyzer tracks the contract deployment according to the contract hash containing the compilation and interaction information; 3. The dynamic analyzer uses the hash information to get the contract ABI and interact with the contract.

Implicit Deployed Contract Bridging*:* The implicitly deployed contract is the contract that is deployed by the other contract but not by the deployer. This is a common practice in Dapp development for better modularity and scalability. Traditional tools cannot discover the implicitly deployed contract and thus cannot interact with these contracts.

To solve this problem, we integrate the Contract Mocker in the dynamic analyzer in the Eye module in figure 1. The Contract Mocker is used to automatically mock the implicit deployed contract so the system can interact with the mocked one, which is bridged to the real one. The workflow of this can be described as: 1. When the contracts are deployed, the implicit deployed contract is declared and the ABI is extracted; 2. The Contract Mocker automatically mocks the implicit deployed contract according to the ABI; 3. The system interacts with the mocked contract, transferring the transaction to the real contract.

VI. EVALUATIONS

A. Implementation

The experiment is conducted on the local desktop machine with the following configuration: Intel 12th Gen i5-12600K CPU 3.70GHz, 32GB RAM, and Nvidia RTX 3070Ti GPU, Ubuntu 20.04 LTS OS with WSL 2.0. The method is implemented in Python 3.6, and the main libraries used are brownie, truffle, ganache, and go-Ethereum. The system is tested on the 15K+ smart contracts and labeled datasets for correctness validation and performance evaluation.

B. Benchmark and Datasets

The dataset comprises 2 parts: the training dataset and the test dataset. The training dataset contains 231 contracts codebase and corresponding auditing reports, collected from the popular smart contract auditing platforms, including Immunefi[19], Open-Zeppelin[40], PeckShield[31] and Code4Rena[7]. The training dataset is used for brain and intuition model training.

The test dataset is collected from Etherscan, the most popular Ethereum contract explorer, and includes 15K+ real-life smart contracts. It is divided into two parts: the dataset labeled with vulnerabilities for validation and performance evaluation and the fresh smart contracts dataset for novel vulnerability discovery. The dataset for validation contains 556 contracts labeled manually, including 418 traditional types and 138 novel types. The traditional types include 87 Unhandled Exception, 80 Reentrance, 89 Suicidal, 77 Block State Dependency, and 86 Leaking. The novel types include 42 Transfer Arrival, 32 Total Fund Conservation, 21 Interest Rate Stability, and 43 Value Stability. This dataset is used for correctness validation and performance evaluation. The fresh smart contracts dataset contains 15K+ real-life smart contracts used for novel vulnerability discovery. They are collected from the Etherscan[8], including 23 big Dapp leveled applications(because of the complexity of deployment, we only choose the most focused Dapps. We could expand the dataset in the future) and 15,132 small contracts. Thus, we can embody our capability in big contracts and broaden the testing scope as much as possible.

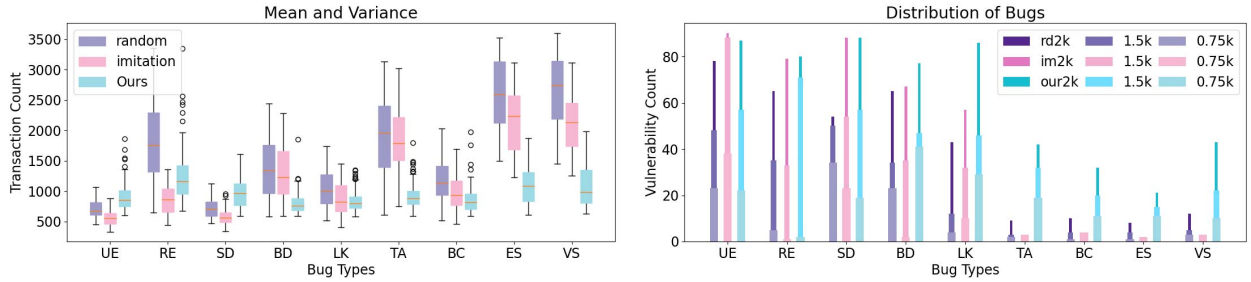
C. Statistics Analysis

In 15,132 real-life contracts, we found 213 money-related vulnerabilities in small smart contracts, and in 25 Dapp leveled contracts, we found 4 contracts with money-related vulnerabilities. They are 100% triggered in our locally configured test net, but because of the complexity of the main net, some of them might not be successfully triggered. However, we still consider them to be important bugs, so we have tried to contact the contract developers. And for the contracts that cannot find the author, we have reported them to the Ethereum Platform.

1) ***4 Vulnerabilities in Big Contracts****:* In the 20 Dapp leveled contracts, we found 4 contracts with money-related vulnerabilities. They are listed in the table III.

0x96cc...2fc8*:* This contract is a component of a lending protocol. The vulnerability exists in one of its payable functions, which allows the user to deposit the token to the contract while the contract does not update the total deposit amount or the user's share correctly. Our fuzzer tracked the vulnerability by

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.



(a) Bug Discovery Time Measured in Transactions Count

(b) Bug Count in Limited Time

Fig. 2: Coverage Performance. *The figure 2a is a box plot where the x-axis is the bug type, and the y-axis is the bug discovery time measured in transactions count. The red bar in the box is the mean value, the top and bottom of the box are the 75% and 25% quantile, the top and bottom of the whisker are the 95% and 5% quantile, and the red point is the outlier. Figure 2b is a bar plot where the x-axis is the bug type, and the y-axis is the bug count in a limited time. It uses different shades of color & width to represent the different time limitations; the darker & thinner the bar, the longer the time limitation. For both figures, the blue bar is the FuzzLaPro, the pink bar is the ILF, and the purple bar is the random fuzzer.*

Contract Address	Vulnerability Type				Severity
	TA	BC	ES	VS	
0x96cc...2fc8	✓	✓	-	-	medium
0x9cc8...b852	-	✓	-	-	high
0x18b7...7f38	-	-	✓	-	high
0x7b74...9020	-	-	✓	✓	high

TABLE III: Novel Vulnerability Samples In Big Contracts

depositing the token directly to the contract and checking out the 'Transfer Arrival' and 'Balance Conservation' violations.

It seems to be a corny "Unexpected Ether" vulnerability. Unfortunately, the tokens affected are not Ether, and traditional tools cannot detect them. The careless users' token will be lost in the contract, but considering it will not cause further damage, we consider it as medium severity.

0x9cc8...b852: This contract contains a mint function, which is not guarded by the '`onlyOwner`' modifier but protected by the required statement. The statement verifies the variable '`tx.origin`' is equal to the owner's address. Our fuzzer tacked the vulnerability by calling the mint function from the owner's address but detouring the transaction from the attacker's contract. Thus, the 'Balance Conservation' is violated, and the attacker's contract can mint the token infinitely. Although the condition is strict, the vulnerability is still critical, so we consider it as high severity.

0x18b7...7f38: This contract is a DeFi protocol, which contains a function to update the exchange rate and the interest rate of the token. The vulnerability exists in one of its payable functions, similarly to the first contract, the contract allow direct token transfer to the contract, but the contract does not update the local state correctly. What is worse, the contract calculates the interest rate and the exchange rate based on both the local state and the transferred amount, thus the attacker can manipulate the interest rate and the exchange rate with very small cost. Our fuzzer tracked the vulnerability by depositing the token directly to the contract and then locally borrowing the token, and then the 'Exchange Rate Stability' was violated. The vulnerability can corrupt the whole project, so we consider it high severity.

0x7b74...9020: This contract is a DeFi protocol, where the user can deposit the token to the contract and borrow other tokens with collateral or deposit tokens for accruing interest. The vulnerability is tracked by the fuzzer by direct token transfer, which then borrows the token, thus increasing the value of the collateral token for borrowing more or accruing more interest. 'Interest Rate Stability' and 'Value Stability' are violated. The vulnerability can cause the contract insolvent, so we consider it high severity.

2) ***213 Vulnerabilities in Small Contracts:*** For the smaller contracts dataset, we found 213 money-related vulnerabilities from the 15,132 real-life smart contracts. Compared to the traditional tools, the FuzzLaPro only focuses on the real financial critical vulnerabilities, so the number of vulnerabilities is less but more informative. The vulnerability types and severity distribution are shown in the table IV.

	Vulnerability Type				Severity
	TA	BC	ES	VS	
	34	65	0	0	low
Count	23	43	14	8	medium
	7	12	3	4	high
totality	64	120	17	12	

TABLE IV: Novel Vulnerability In Small Contracts Statistics

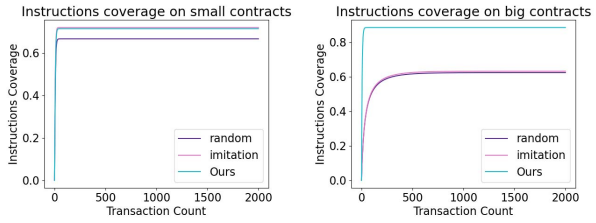
In this table, we can see that the "Transfer Arrival" and "Balance Conservation" are the most common types of vulnerabilities, and the "Interest Rate Stability" and "Value Stability" are the most critical types of vulnerabilities. We have reported all these vulnerabilities to the Ethereum Platform.

3) ***Mitigation:*** All the vulnerabilities above we found are 100% triggered in our locally configured test net, but because of the complexity of the main net, some might not be successfully triggered. Considering the cost of testing on the main net, we chose to write the PoC and tried to contact the contract developers. For those we cannot find the author, we have reported to the Ethereum Platform.

4) ***Instruction Coverage Performance:*** The coverage performance evaluation dataset is the 15K+ real-life smart contracts test dataset we mentioned in section VI-B. To evaluate the

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

performance of the FuzzLaPro, we compare the coverage performance of the FuzzLaPro with the SoTA AI-driven fuzzer ILF[13] and the random fuzzer. The coverage performance is measured by average instruction coverage on the small contracts or the big contracts. The results are shown in the figure 3.



(a) Coverage on Small Contracts **(b)** Coverage on Big Contracts

Fig. 3: Coverage Performance. *The blue line is the FuzzLaPro, the pink line is the imitation strategy of the ILF, and the purple line is the random fuzzer. In figure 3a, the FuzzLaPro and the ILF are mostly overlapped; the ILF performs slightly better, and the random fuzzer performs the worst. In the figure 3b, the FuzzLaPro outperforms the ILF and the random fuzzer.*

In the figure 3a, the FuzzLaPro and the ILF have similar performance on the small contracts, but the FuzzLaPro performs better on the big contracts. The random fuzzer has the worst performance. Thanks to the effort of adapting the EVM version compatibility and the implicit deployed contract bridging, the FuzzLaPro outperforms the ILF and the random fuzzer on the big contracts.

5) ***Vulnerability Discovering Performance****:* The vulnerability discovering performance evaluation dataset is the 556 labeled dataset, the validation dataset as a part of the 15K+ real-life smart contracts test dataset mentioned in the section VI-B. We still use the ILF[13] and the random fuzzer as the baseline to evaluate the performance on discovering the vulnerabilities. We measure the average time to trigger the vulnerability on different types of vulnerabilities in figure 2a, and the number of vulnerabilities triggered in limited time in figure 2b.

To implement the evaluation, we replace the ILF's detection module and the random fuzzer with ours while keeping the other modules the same. The results should be fair. For the bug discovery time in figure 2a, except the "Unexpected Ethe," "Reentrancerance," and "Self Destruct" vulnerabilities, the FuzzLaPro consistently outperforms the ILF and the random fuzzer. The ILF[13] is a state-of-the-art AI-driven fuzzer specializing in traditional vulnerabilities, but the FuzzLaPro specializes more in financial vulnerabilities. Overall, the FuzzLaPro performed better during bug discovery time. For the bug count in limited time in figure 2b, the FuzzLaPro also outperforms the ILF and the random fuzzer except the "Unexpected EtheReentrancerance", and "Self Destruct" vulnerabilities. FuzzLaPro performs better on our new types of vulnerabilities, "Transfer Arrival," "Total Fund Conservation," "Interest Rate Stability," and "Value Stability."

After the performance evaluation, the FuzzLaPro had an equal or better performance on the coverage and generally better performance on the vulnerability discovery. As the conclusion of the general practicability, the FuzzLaPro is more informative and helpful than the traditional fuzzers on the monetary exploit vulnerabilities, surprisingly with excellent performance.

VII. Related Work

Extensive efforts have been made to mitigate vulnerabilities in the landscape of smart contract security, particularly within Ethereum smart contracts. These include deploying security tools, launching bug bounty programs, and establishing auditing companies. Traditional tools have struggled with high false positive rates and low efficiency, making manual review essential despite its high cost.

Several tools and methodologies have been developed targeting common vulnerabilities listed by the SWC[9], an official record maintained by the Ethereum Foundation. Notable tools include Zeus[21], Mythril[28], and Oyente[27], which primarily utilize opcode and stack-based pattern matching. These tools are complemented by dynamic analyzers like fuzzers (e.g., Echidna[12] and Slither[10]) and formal verification tools (e.g., F*[3] and Isabelle/HOL[1]), although the latter suffer from scalability issues due to the need for highly specific verification statements.

Recent innovations in smart contract security also explore the integration of machine learning with traditional tools to improve efficiency and accuracy. For instance, the Smarter-Contract and ILF utilize machine learning to assist fuzzers by focusing on the most critical code branches[14, 33]. Another innovative approach involves using computer vision models to analyze compiled bytecode images to assist decision-making in fuzzers[22].

Despite these advancements, a gap remains in tools that can comprehensively understand the logic of smart contracts, mimicking human auditor expertise. Our work, FuzzLaPRO, seeks to bridge this gap by employing expert auditing reports to train NLP models that assist fuzzers, aligning closely with human-like understanding and reasoning capabilities.

VIII. Conclusion

Our research has successfully addressed the sophisticated nature of financial vulnerabilities in smart contracts through the development of formalized fuzzer rules. The implementation and subsequent evaluation of these rules have proven their effectiveness, marking a significant advancement over existing tools. Future work will focus on enhancing the usability of our system and developing a comprehensive suite of practical tools and theoretical frameworks specifically aimed at combating monetary exploit vulnerabilities. This endeavor will not only refine our understanding of smart contract security but also significantly contribute to reducing the potential financial risks in the blockchain domain.

References

[1] S. Amani, M. Bortin, M. Bégel, and M. Staples, "Towards verifying ethereum smart contract bytecode in Isabelle/HOL," *CPP 2018 - Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs, Co-located with POPL 2018*, vol. 2018-January, pp. 66–77, Jan. 2018, publisher: Association for Computing Machinery, Inc ISBN: 9781450355865.

[2] K. Babel, P. Daian, M. Kelkar, and A. Juels, "Clockwork Finance: Automated Analysis of Economic Security in Smart

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

Contracts," in *2023 IEEE Symposium on Security and Privacy (SP)*, May 2023, pp. 2499–2516, iSSN: 2375-1207. [Online]. Available: https://ieeexplore.ieee.org/document/10179346

[3] K. Bhargavan, A. Delignat-Lavaud, C. Fournet, A. Gollamudi, G. Gonthier, N. Kobeissi, N. Kulatova, A. Rastogi, T. Sibut-Pinote, N. Swamy, and S. Zanella-Béguelin, "Formal verification of smart contracts: Short paper," *PLAS 2016 - Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security, co-located with CCS 2016*, pp. 91–96, Oct. 2016, publisher: Association for Computing Machinery, Inc ISBN: 9781450345743.

[4] S. Chaliasos, M. A. Charalambous, L. Zhou, R. Galanopoulou, A. Gervais, D. Mitropoulos, and B. Livshits, "Smart Contract and DeFi Security Tools: Do They Meet the Needs of Practitioners?" in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. Lisbon Portugal: ACM, Feb. 2024, pp. 1–13. [Online]. Available: https://dl.acm.org/doi/10.1145/3597503.3623302

[5] M. Cho, S. Kim, and T. Kwon, "Intriguer: Field-level constraint solving for hybrid fuzzing," *Proceedings of the ACM Conference on Computer and Communications Security*, pp. 515–530, Nov. 2019, publisher: Association for Computing Machinery ISBN: 9781450367479.

[6] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, and S. K. Cha, "SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses," *Proceedings - 2021 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021*, pp. 227–239, 2021, publisher: IEEE ISBN: 9781665403375.

[7] Code4rena, "Code4rena," 2024. [Online]. Available: https://code4rena.com/

[8] Ethereum, "Ethereum Virtual Machine (EVM)," 2023. [Online]. Available: https://ethereum.org

[9] Ethereum Foundation, "Smart Contract Weakness Classification (SWC)," 2024. [Online]. Available: https://swcregistry.io/

[10] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," *Proceedings - 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, WETSEB 2019*, pp. 8–15, 2019, arXiv: 1908.09878 ISBN: 9781728122571.

[11] P. Godefroid, H. Peleg, and R. Singh, "Learn&Fuzz: Machine learning for input fuzzing," *ASE 2017 - Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, pp. 50–59, Nov. 2017, arXiv: 1701.07232 Publisher: Institute of Electrical and Electronics Engineers Inc. ISBN: 9781538626849.

[12] A. Groce and G. Grieco, "Echidna-parade: A tool for diverse multicore smart contract fuzzing," *ISSTA 2021 - Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 658–661, 2021, iSBN: 9781450384599.

[13] D. He, Z. Deng, Y. Zhang, S. Chan, Y. Cheng, and N. Guizani, "Smart Contract Vulnerability Analysis and Security Audit," *IEEE Network*, vol. 34, no. 5, pp. 276–282, Sep. 2020, conference Name: IEEE Network. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9143290

[14] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," *Proceedings of the ACM Conference on Computer and Communications Security*, pp. 531–548, Nov. 2019, publisher: Association for Computing Machinery ISBN: 9781450367479. [Online]. Available: https://dl.acm.org/doi/10.1145/3319535.3363230

[15] Immunefi, "Balancer Rounding Error Bugfix Review," Oct. 2023. [Online]. Available: https://medium.com/immunefi/balancer-rounding-error-bugfix-review-cbf69482ee3d

[16] ——, "Fei Protocol Flashloan Vulnerability Bugfix Review," Feb. 2023. [Online]. Available: https://medium.com/immunefi/fei-protocol-flashloan-vulnerability-postmortem-7c5dc001affb

[17] ——, "Silo Finance Logic Error Bugfix Review," Jun. 2023. [Online]. Available: https://medium.com/immunefi/silo-finance-logic-error-bugfix-review-35de29bd934a

[18] ——, "Yield Protocol Logic Error Bugfix Review," Jul. 2023. [Online]. Available: https://medium.com/immunefi/yield-protocol-logic-error-bugfix-review-7b86741e6f50

[19] ——, "Immunefi," 2024. [Online]. Available: https://immunefi.com/

[20] ——, "Immunefi," 2024. [Online]. Available: https://immunefi.medium.com

[21] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "Zeus: analyzing safety of smart contracts." in *Ndss*, San Diego, CA, 2018, pp. 1–12.

[22] B. Kolosnjaji, G. Eraisha, G. Webster, A. Zarras, and C. Eckert, "Empowering convolutional networks for malware classification and analysis," *Proceedings of the International Joint Conference on Neural Networks*, vol. 2017-May, pp. 3838–3845, Jun. 2017, publisher: Institute of Electrical and Electronics Engineers Inc. ISBN: 9781509061815.

[23] A. Kumar, C. Fischer, S. Tople, and P. Saxena, "A traceability analysis of Monero's blockchain," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 10493 LNCS, pp. 153–173, 2017, iSBN: 9783319663982.

[24] Z. Li, S. Lu, R. Zhang, R. Xue, W. Ma, R. Liang, Z. Zhao, and S. Gao, "SmartFast: an accurate and robust formal analysis tool for Ethereum smart contracts," *Empirical Software Engineering*, vol. 27, no. 7, Dec. 2022, publisher: Springer.

[25] Z. Liu, P. Qian, J. Yang, L. Liu, X. Xu, Q. He, and X. Zhang, "Rethinking Smart Contract Fuzzing: Fuzzing With Invocation Ordering and Important Branch Revisiting," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 1237–1251, Jan. 2023, publisher: Institute of Electrical and Electronics Engineers (IEEE).

[26] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction," Aug. 2018, arXiv:1808.09602 [cs]. [Online]. Available: http://arxiv.org/abs/1808.09602

[27] L. Luu, D. H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," *Proceedings of the ACM Conference on Computer and Communications Security*, vol. 24-28-October-2016, pp. 254–269, Oct. 2016, publisher: Association for Computing Machinery ISBN: 9781450341394.

[28] F. Ma, Z. Xu, M. Ren, Z. Yin, Y. Chen, L. Qiao, B. Gu, H. Li, Y. Jiang, and J. Sun, "Pluto: Exposing Vulnerabilities in Inter-Contract Scenarios," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4380–4396, Nov. 2022, publisher: Institute of Electrical and Electronics Engineers Inc.

[29] M. Möser, K. Soska, E. Heilman, K. Lee, H. Heffan, S. Srivastava, K. Hogan, J. Hennessey, A. Miller, A. Narayanan, and N. Christin, "An Empirical Analysis of Traceability in the Monero Blockchain," *Proceedings on Privacy Enhancing Technologies*, vol. 2018, no. 3, pp. 143–163, 2018, arXiv: 1704.04299.

[30] M. Ndiaye and K. Konaté, "Formal Analysis of Smart Contracts: Model Impact Factor on Criminality," *Lecture Notes in Networks and Systems*, vol. 320 LNNS, pp. 3–13, 2022, publisher: Springer Science and Business Media Deutschland GmbH ISBN: 9783030861612.

[31] PeckShield, "Uniswap/Lendf.Me Hacks: Root Cause and Loss Analysis | by PeckShield | Medium," 2023. [Online]. Available: https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09

[32] G. Roşu and T. F. Şerbănuţă, "An overview of the K semantic framework," *Journal of Logic and Algebraic Programming*, vol. 79, no. 6, pp. 397–434, Aug. 2010.

[33] C. Sendner, H. Chen, H. Fereidooni, L. Petzi, J. König, J. Stang, A. Dmitrienko, A.-R. Sadeghi, and F. Koushanfar, "Smarter Contracts: Detecting Vulnerabilities in Smart Contracts with Deep Transfer Learning," in *Proceedings 2023 Network and Distributed System Security Symposium*. San Diego, CA, USA: Internet Society, 2023. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2023/02/ndss2023_s263_paper.pdf

[34] S. Smolka, J.-R. Giesen, P. Winkler, O. Draissi, L. Davi, G. Karame, and K. Pohl, "Fuzz on the Beach: Fuzzing Solana Smart Contracts," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 1197–1211. [Online]. Available: https://dl.acm.org/doi/10.1145/3576915.3623178

[35] J. Su, H.-N. Dai, L. Zhao, Z. Zheng, and X. Luo, "Effectively Generating Vulnerable Transaction Sequences in Smart Contracts with Reinforcement Learning-guided Fuzzing," pp. 1–12, Oct. 2022, publisher: Association for Computing Machinery (ACM) ISBN: 9781450396240.

[36] N. Sánchez-Gómez, L. Morales-Trujillo, J. J. Gutiérrez, and J. Torres-

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.

Valderrama, "The Importance of Testing in the Early Stages of Smart Contract Development Life Cycle," *JWE*, Jun. 2020. [Online]. Available: https://journals.riverpublishers.com/index.php/JWE/article/view/3357

[37] C. Team, "Stolen Crypto Falls in 2023, but Hacking Remains a Threat," Jan. 2024. [Online]. Available: https://www.chainalysis.com/blog/crypto-hacking-stolen-funds-2024/

[38] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," *Proceedings of the ACM Conference on Computer and Communications Security*, pp. 67–82, Oct. 2018, arXiv: 1806.01143 Publisher: Association for Computing Machinery ISBN: 9781450356930.

[39] J. Wang, B. Chen, L. Wei, and Y. Liu, "Skyfire: Data-Driven Seed Generation for Fuzzing," *Proceedings - IEEE Symposium on Security and Privacy*, pp. 579–594, Jun. 2017, publisher: Institute of Electrical and Electronics Engineers Inc. ISBN: 9781509055326.

[40] S. Wesley, M. Christakis, J. A. Navas, R. Trefler, V. Wüstholz, and A. Gurfinkel, "Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13182 LNCS, pp. 425–449, 2022, publisher: Springer Science and Business Media Deutschland GmbH ISBN: 9783030945824.

[41] S. Wu, L. Wu, Y. Zhou, R. Li, Z. Wang, X. Luo, C. Wang, and K. Ren, "Time-Travel Investigation: Toward Building a Scalable Attack Detection Framework on Ethereum," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 3, Jul. 2022, arXiv: 2005.08278 Publisher: Association for Computing Machinery.

[42] M. Zhang, X. Zhang, Y. Zhang, and Z. Lin, "TXSPECTOR: Uncovering attacks in ethereum from transactions," *Proceedings of the 29th USENIX Security Symposium*, pp. 2775–2792, 2020, iSBN: 9781939133175.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 20:55:59 UTC from IEEE Xplore. Restrictions apply.